CloudGuard Security Scanner

For the September review fixes and deployment requirements, see the upgrade notes.

A comprehensive, containerized security scanning platform designed to identify vulnerabilities across cloud, Kubernetes, infrastructure-as-code, and application assets. The platform can run on DigitalOcean infrastructure while still scanning AWS, GCP, Kubernetes, and IaC targets when users provide credentials or files.

Features

- **Multi-Cloud Support**: Scans AWS, GCP, Kubernetes clusters, and IaC files.
- **DigitalOcean/E2E-Friendly Runtime**: Deploy the platform with PostgreSQL, worker containers, and S3-compatible evidence storage. Public HTTPS/routing should be handled by Dokploy/Traefik or another external reverse proxy.
- **Web App Scanning**: Integrated OWASP ZAP and Nuclei for web vulnerability detection.
- **Secret Scanning**: Detects hardcoded secrets using Gitleaks.
- **Dependency Analysis**: Checks for vulnerable dependencies with Trivy and Safety.
- **Unified Dashboard**: View all findings in a single, easy-to-use web interface.
- **Reason-and-Act Workflow**: Adds tenant-scoped risk context, contextual prioritization, sandbox validation proof, remediation approval modes, and STRIDE threat models without depending on one cloud provider.
- **Temporary Sandbox Labs**: Creates short-lived vulnerable IaC/AWS/GCP/Kubernetes demo labs inside isolated sandbox environments, records proof as GRC evidence, queues scans, and auto-destroys resources after scan completion or TTL expiry.
- **GRC Evidence Ingestion**: Accepts normalized evidence from decoupled scanners/connectors and maps it back to compliance controls.
- **Merged GRC Platform Module**: Includes the handoff GRC platform under `grc-platform/` for compliance workflows, evidence review, risk management, audit workflows, and CloudGuard connector visualization.
- **Dockerized**: Fully containerized for easy deployment.

Prerequisites

Before you begin, ensure you have the following installed: - Docker Desktop (includes Docker Compose) - Git

Installation & Setup

1. Clone the Repository

```
git clone <your-repo-url>
cd Cloud_Vulnerabilities
```

2. Configure Environment Variables

Create a `.env` file in the root directory. You can start by copying the example (if available) or ensuring you have the following keys:

```
# Database (PostgreSQL)
POSTGRES_USER=scanner_user
POSTGRES_PASSWORD=scanner_pass
POSTGRES_DB=scanner_db

# Redis
REDIS_PASSWORD=redis_password

# Application Secrets
SECRET_KEY=your-super-secret-key-change-this
ENCRYPTION_KEY=your-encryption-key

# Worker / Queue
QUEUE_BACKEND=postgres
WORKER_TOKEN=your-worker-token
SCAN_JOB_INLINE_WORKER=false

# Temporary sandbox labs
SANDBOX_LABS_ENABLED=true
SANDBOX_LAB_INLINE_WORKER=false
SANDBOX_IAC_DEPLOY=true
SANDBOX_AWS_DEPLOY=false
SANDBOX_GCP_DEPLOY=false
SANDBOX_KUBERNETES_DEPLOY=false

# Evidence storage on DigitalOcean Spaces
OBJECT_STORAGE_PROVIDER=digitalocean-spaces
OBJECT_STORAGE_ENDPOINT_URL=https://blr1.digitaloceanspaces.com
OBJECT_STORAGE_REGION=blr1
OBJECT_STORAGE_BUCKET=cloudguard-evidence
OBJECT_STORAGE_ACCESS_KEY_ID=your_spaces_key
OBJECT_STORAGE_SECRET_ACCESS_KEY=your_spaces_secret

# GCP Configuration (Optional)
GCP_PROJECT_ID=your_project_id
GCP_SERVICE_ACCOUNT_JSON=/app/config/gcp-service-account.json

# AI Analysis (Optional OpenAI-compatible endpoint)
AI_BASE_URL=http://your-model-host:11434/v1
AI_MODEL=llama3.1
```

```
AI_API_KEY=local-model
OPENAI_API_KEY=your_openai_key

# Public Deployment
DOMAIN=your-domain.com
```

3. Setup Cloud Permissions For Scan Targets

To scan a cloud account, add read-only credentials in the UI. AWS scanning still uses IAM/CloudFormation for target account access, but CloudGuard itself does not need to be hosted on AWS. 1. Navigate to the `infra/` folder. 2. Use the provided CloudFormation template or `iam_permissions.json` to configure an IAM Role or User with the necessary permissions. - See `infra/README.md` for detailed instructions on deploying the permissions stack.

Usage

Start the Application

Run the entire stack using Docker Compose:

```
docker compose --profile dev up --build
```

This will start: - **Backend API**: http://localhost:8000 - **Frontend Dashboard**: http://localhost:8000 - **Database & Redis**: Background services

Access the Dashboard

Open your browser and navigate to: **http://localhost:8000**

Running a Scan

1. Log in to the dashboard.
2. Configure your target (URL, IP, or Cloud Account).
3. Click **"New Scan"**.
4. View real-time progress and detailed reports.

Architecture

- **Frontend**: HTML/JS Dashboard (served via Nginx)
- **Backend**: FastAPI (Python)
- **Database**: PostgreSQL (Findings storage)
- **Queue**: PostgreSQL-backed `scan_jobs`
- **Evidence/GRC Layer**: `/api/evidence`, `/api/compliance/*`, `/api/risk/*`, `/api/validation/*`, `/api/remediation/*`, and `/api/threat-models/*`
- **Sandbox Lab Layer**: `/api/sandbox-labs/*` plus `sandbox-lab-worker` for temporary vulnerable lab deployment, scan proof, and cleanup.

- **Reason & Act UI**: `/reason-act` for validation jobs, remediation actions, threat models, and workflow visibility.
- **Full GRC Platform**: `grc-platform/` runs as a sibling governance platform. Set `GRC_PLATFORM_URL` so the CloudGuard `/grc` entry point opens the hosted GRC dashboard.
- **Tools**: Trivy, Gitleaks, Nuclei, CloudFox, OWASP ZAP

See `docs/vendor-neutral-reason-act-architecture.md` for the workflow that maps discovery, prioritization, validation, remediation, and threat modeling into the decoupled scanner architecture. See `docs/sandbox-lab-workflow.md` for the temporary vulnerable lab setup. See `docs/grc-platform-merge.md` for how the merged GRC platform fits into the latest CloudGuard project.

Security Note

Never commit your `.env` file. It contains sensitive API keys and secrets. This repository includes a `.gitignore` to prevent accidental commits, but always double-check.

Deploy Online (Production + HTTPS)

The production app no longer starts its own Caddy reverse proxy. Use the Dokploy deployment for public hosting, or put an external reverse proxy in front of the backend service.

For DigitalOcean, use the deployment guide in `deploy/digitalocean/README.md`:

```
cp .env.digitalocean.example .env
docker compose -f deploy/digitalocean/docker-compose.yml --env-file .env up -d --build
```

1. DNS + Firewall

- Point your domain `A` record to your server public IP.
- Open inbound ports `80` and `443` only on the reverse proxy host, for example Dokploy/Traefik.

2. Set deployment env vars

In your `.env` (or `.env.prod`) set: - `DOMAIN=your-domain.com` - `BACKEND_BIND_ADDRESS=127.0.0.1` - `BACKEND_PORT=8000` - `SECRET_KEY`, `ENCRYPTION_KEY`, cloud credentials, DB/Redis passwords

3. Start production stack

```
docker compose --profile prod up -d --build postgres redis backend
```

4. Verify

```
docker compose --profile prod ps
curl -I http://localhost:8000/health
```

5. Security defaults included

- PostgreSQL port bound to localhost only.
- Redis port bound to localhost only.
- Backend production port bound to localhost by default.
- Public HTTPS and domain routing are expected to be handled by Dokploy/Traefik or another reverse proxy.